

PART IIB REVISION NOTES

4F10: Deep Learning and Structured Data

Qianshuo (Harry) Ye

April 2026

1 Classifiers

Bayes's decision rule: for a classifier $f(\mathbf{x}^*, \boldsymbol{\theta})$, it generates a decision $\hat{y} \in \{\omega_1, \dots, \omega_K\}$ by minimising the expected loss:

$$\hat{y} = \arg \min_{\omega} \left\{ \sum_{i=1}^K \mathcal{L}(\omega, \omega_i) P(\omega_i | \mathbf{x}^*) \right\}$$

For binary classification, the decision rule is $\hat{y} = \arg \max_{\omega \in \{\omega_1, \omega_2\}} P(\omega | \mathbf{x}^*)$.

Training a classifier: to train a classifier, minimise the expected loss

$$\mathcal{L}_{\text{act}} = \int \left[\sum_{i=1}^K \mathcal{L}(f(\mathbf{x}, \boldsymbol{\theta}), \omega_i) P(\omega_i | \mathbf{x}) \right] p(\mathbf{x}) d\mathbf{x}$$

which can be approximated as the empirical loss

$$\mathcal{L}_{\text{emp}} = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f(\mathbf{x}, \boldsymbol{\theta}), y_i)$$

where $\mathbf{x}_i \sim p(\mathbf{x})$ and $y_i \sim P(\omega | \mathbf{x}_i)$ are (observation, class label) pairs in a supervised training data $\mathcal{D}$. In the limit $N \rightarrow \infty$, $\mathcal{L}_{\text{emp}} = \mathcal{L}_{\text{act}}$ but in practice $\mathcal{L}_{\text{emp}} \leq \mathcal{L}_{\text{act}}$.

To assess the performance on unseen data, split data into *training* and *evaluation data*.

Probability of error: For a binary classification problem, the probability of error is the probability that a sample from class ω_1 is misclassified as ω_2 plus the probability that a sample from class ω_2 is misclassified as ω_1 :

$$P(\text{error}) = \int_{\Omega_2} p(\omega_1, \mathbf{x}) d\mathbf{x} + \int_{\Omega_1} p(\omega_2, \mathbf{x}) d\mathbf{x}$$

where Ω_1 and Ω_2 are the decision regions for class ω_1 and ω_2 respectively.

Generative vs Discriminative models:

Generative models model the joint distribution of observations and classes $p(\mathbf{x}, \omega | \boldsymbol{\theta})$, and the posterior can be obtained from Bayes' rule:

$$P(\omega_i | \mathbf{x}^*, \boldsymbol{\theta}) = \frac{p(\mathbf{x}^*, \omega_i | \boldsymbol{\theta})}{\sum_{j=1}^K p(\mathbf{x}^*, \omega_j | \boldsymbol{\theta})} = \frac{P(\omega_i) p(\mathbf{x}^* | \omega_i, \boldsymbol{\theta})}{\sum_{j=1}^K P(\omega_j) p(\mathbf{x}^* | \omega_j, \boldsymbol{\theta})}$$

where $P(\omega_i)$ is the prior probability of class ω_i .

- Strength: Useful when there is a need to generate data or handle missing data.
- Limitation: assumptions about the conditional distribution and prior might not hold.
- Estimation: Find parameters θ that maximise the likelihood of the observations given labels, where a separate model is trained for each class ω_i :

$$\hat{\theta}_i = \arg \max_{\theta} \left\{ \sum_{j: y_j = \omega_i} \log P(\mathbf{x}_j | \omega_i, \theta) \right\}$$

To estimate the prior, a simple way is to count the number of samples in each class $P(\omega_i) = \frac{N_i}{N}$, or add 1 to all classes $P(\omega_i) = \frac{N_i+1}{N+K}$.

- Examples: Naive Bayes, Gaussian mixture model, hidden Markov model.

Discriminative models directly model the posterior $P(\omega | \mathbf{x}^*, \theta)$.

- Strength: Achieve better results as they focus directly on the decision boundary.
- Limitation: lack the generative capability.
- Estimation: Find parameters θ that maximise the likelihood of the labels given observations:

$$\hat{\theta} = \arg \max_{\theta} \left\{ \sum_{i=1}^N \log P(y_i | \mathbf{x}_i, \theta) \right\}$$

- Examples: logistic regression, support vector machine.

Multivariate Gaussian class conditional PDF: For a d -dimensional observation $\mathbf{x}$, the class conditional PDF is

$$p(\mathbf{x} | \omega_i, \theta_i) = \frac{1}{(2\pi)^{d/2} |\Sigma_i|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_i)^T \Sigma_i^{-1} (\mathbf{x} - \boldsymbol{\mu}_i) \right)$$

The maximum likelihood estimates of the mean and covariance are

$$\begin{aligned} \hat{\boldsymbol{\mu}}_i &= \frac{1}{N_i} \sum_{j: y_j = \omega_i} \mathbf{x}_j \\ \hat{\Sigma}_i &= \frac{1}{N_i} \sum_{j: y_j = \omega_i} (\mathbf{x}_j - \hat{\boldsymbol{\mu}}_i)(\mathbf{x}_j - \hat{\boldsymbol{\mu}}_i)^T \end{aligned}$$

Decision boundaries: The decision boundary is the set of points $\mathbf{x}$ where the posterior probabilities of two classes are equal

$$\log(P(\omega_1 | \mathbf{x}, \theta)) = \log(P(\omega_2 | \mathbf{x}, \theta))$$

For a multivariate Gaussian class conditional PDF, the decision boundary is a quadratic function of $\mathbf{x}$:

$$\mathbf{x}^T (\Sigma_1^{-1} - \Sigma_2^{-1}) \mathbf{x} + 2(\Sigma_2^{-1} \boldsymbol{\mu}_2 - \Sigma_1^{-1} \boldsymbol{\mu}_1)^T \mathbf{x} + \boldsymbol{\mu}_1^T \Sigma_1^{-1} \boldsymbol{\mu}_1 - \boldsymbol{\mu}_2^T \Sigma_2^{-1} \boldsymbol{\mu}_2 + \log \left(\frac{|\Sigma_1|}{|\Sigma_2|} \right) - 2 \log \left(\frac{P(\omega_1)}{P(\omega_2)} \right) = 0$$

The decision boundary is linear if $\Sigma_1 = \Sigma_2$.

2 Graphical Models

Independence: Two random variables X and Y are independent ($X \perp Y$) if $P(X, Y) = P(X)P(Y)$.

Conditional independence: Two random variables X and Y are conditionally independent given a third variable Z ($X \perp Y|Z$) if $P(X, Y|Z) = P(X|Z)P(Y|Z)$.

Conditional independence can be used to break down high-dimensional distributions into a product of simple factors evaluated only on a subset of variables.

Graphical models: factorisation based on conditional independence can be represented as a graph, where nodes represent random variables and edges represent dependencies between variables.

Bayesian networks: a directed acyclic graph where edges represent causal relationships between variables. Denoting PA_{X_i} as the set of parent nodes of X_i , the joint distribution can be factorised as $P(X_1, \dots, X_d) = \prod_{i=1}^d P(X_i|\text{PA}_{X_i})$. Denote non-descendants of X_i as ND_{X_i} , there exists conditional independencies ($X_i \perp \text{ND}_{X_i}|\text{PA}_{X_i}$).

Efficient marginalisation in graphical models: For a Bayesian network $A \rightarrow B \rightarrow C \rightarrow D$, the probability of d can be factorised as

$$p(d) = \sum_c \sum_b \sum_a p(d|c)p(c|b)p(b|a)p(a)$$

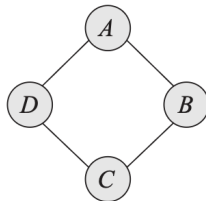
which have cost of $\mathcal{O}(n^4)$ when each variable takes n different values. Reordering gives

$$p(d) = \sum_c p(d|c) \sum_b p(c|b) \sum_a p(b|a)p(a)$$

which have cost of $\mathcal{O}(n^2)$ (given by the largest factor).

Markov networks: an undirected graph where edges represent symmetric relationships between variables. The network is annotated with positive *potential* functions $\phi_1(\mathbf{D}_1), \dots, \phi_k(\mathbf{D}_k)$, where $\{\mathbf{D}_i\}$ is the set of variables in the i -th *clique* – a fully connected set of nodes. The joint distribution can be factorised as $P(X_1, \dots, X_d) = \frac{1}{Z} \prod_{i=1}^k \phi_i(\mathbf{D}_i)$, where Z is the partition function. The network encodes the conditional independencies ($A \perp B|C$), where C separates A from B .

Graph:



Factorization:

$$p(A, B, C, D) = \frac{1}{Z} \phi_1(A, B) \phi_2(B, C) \phi_3(C, D) \phi_4(A, D)$$

Conditional Independencies:

$$(A \perp C|B, D), (B \perp D|A, C)$$

3 Latent Variable and Sequence Models

3.1 Latent Variable Models

Gaussian mixture model: a model where each component is a Gaussian distribution:

$$p(\mathbf{x}) = \sum_{m=1}^M P(c_m) p(\mathbf{x}|c_m) = \sum_{m=1}^M P(c_m) \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_m, \boldsymbol{\Sigma}_m)$$

Factor analysis: a Bayesian network where the observed variable $\mathbf{x}$ is generated from a continuous latent variable $\mathbf{z}$ with lower dimension:

$$p(\mathbf{x}) = \int p(\mathbf{x}|\mathbf{z}) p(\mathbf{z}) d\mathbf{z}$$

If $p(\mathbf{x}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ and $p(\mathbf{z}) = \mathcal{N}(C\mathbf{z}, \boldsymbol{\Sigma}_{\text{diag}})$, $p(\mathbf{x}) = \mathcal{N}(\mathbf{0}, CC^\top + \boldsymbol{\Sigma}_{\text{diag}})$.

Auxiliary function: For Gaussian mixture model, a change of parameters from $\boldsymbol{\theta}^{(k)}$ to $\boldsymbol{\theta}^{(k+1)}$ gives a change of log-likelihood

$$\begin{aligned} \log p(\mathbf{x}|\boldsymbol{\theta}^{(k+1)}) - \log p(\mathbf{x}|\boldsymbol{\theta}^{(k)}) &= \sum_{i=1}^N \log \frac{p(\mathbf{x}_i|\boldsymbol{\theta}^{(k+1)})}{p(\mathbf{x}_i|\boldsymbol{\theta}^{(k)})} \\ &= \sum_{i=1}^N \log \left(\sum_{m=1}^M \frac{P(c_m|\mathbf{x}_i, \boldsymbol{\theta}^{(k)}) p(\mathbf{x}_i, c_m|\boldsymbol{\theta}^{(k+1)})}{P(c_m|\mathbf{x}_i, \boldsymbol{\theta}^{(k)}) p(\mathbf{x}_i|\boldsymbol{\theta}^{(k)})} \right) \\ (\text{Jensen's inequality}) &\geq \sum_{i=1}^N \sum_{m=1}^M P(c_m|\mathbf{x}_i, \boldsymbol{\theta}^{(k)}) \log \left(\frac{p(\mathbf{x}_i, c_m|\boldsymbol{\theta}^{(k+1)})}{p(\mathbf{x}_i, c_m|\boldsymbol{\theta}^{(k)})} \right) \\ &= \mathcal{Q}(\boldsymbol{\theta}^{(k)}, \boldsymbol{\theta}^{(k+1)}) - \mathcal{Q}(\boldsymbol{\theta}^{(k)}, \boldsymbol{\theta}^{(k)}) \end{aligned}$$

where $\mathcal{Q}(\boldsymbol{\theta}^{(k)}, \boldsymbol{\theta}^{(k+1)}) = \sum_{i=1}^N \sum_{m=1}^M P(c_m|\mathbf{x}_i, \boldsymbol{\theta}^{(k)}) \log p(\mathbf{x}_i, c_m|\boldsymbol{\theta}^{(k+1)})$ is known as the *auxiliary function*. For factor analysis, $\mathcal{Q}(\boldsymbol{\theta}^{(k)}, \boldsymbol{\theta}^{(k+1)}) = \sum_{i=1}^N \int p(\mathbf{z}|\mathbf{x}_i, \boldsymbol{\theta}^{(k)}) \log p(\mathbf{x}_i, \mathbf{z}|\boldsymbol{\theta}^{(k+1)}) d\mathbf{z}$.

Expectation maximisation: Given a distribution $q(\mathbf{z}, \tilde{\boldsymbol{\theta}})$, the log-likelihood can be decomposed as

$$\begin{aligned} \mathcal{L}(\boldsymbol{\theta}) &= \log p(\mathbf{x}|\boldsymbol{\theta}) = \int q(\mathbf{z}, \tilde{\boldsymbol{\theta}}) \log p(\mathbf{x}|\boldsymbol{\theta}) d\mathbf{z} \\ &= \int q(\mathbf{z}, \tilde{\boldsymbol{\theta}}) \log \frac{p(\mathbf{x}, \mathbf{z}|\boldsymbol{\theta})}{p(\mathbf{z}|\mathbf{x}, \boldsymbol{\theta})} d\mathbf{z} \\ &= \mathbb{E}_{q(\mathbf{z}, \tilde{\boldsymbol{\theta}})} \left[\log \frac{p(\mathbf{x}, \mathbf{z}|\boldsymbol{\theta})}{p(\mathbf{z}|\mathbf{x}, \boldsymbol{\theta})} \right] \\ &= \mathbb{E}_{q(\mathbf{z}, \tilde{\boldsymbol{\theta}})} \left[\log \frac{p(\mathbf{x}, \mathbf{z}|\boldsymbol{\theta})}{q(\mathbf{z}, \tilde{\boldsymbol{\theta}})} \right] + \underbrace{\int q(\mathbf{z}, \tilde{\boldsymbol{\theta}}) \log \frac{q(\mathbf{z}, \tilde{\boldsymbol{\theta}})}{p(\mathbf{z}|\mathbf{x}, \boldsymbol{\theta})} d\mathbf{z}}_{\text{KL divergence, non-negative}} \\ &\geq \mathbb{E}_{q(\mathbf{z}, \tilde{\boldsymbol{\theta}})} \left[\log \frac{p(\mathbf{x}, \mathbf{z}|\boldsymbol{\theta})}{q(\mathbf{z}, \tilde{\boldsymbol{\theta}})} \right] = \mathcal{F}(q(\mathbf{z}, \tilde{\boldsymbol{\theta}}), \boldsymbol{\theta}) \end{aligned}$$

The EM algorithm iteratively maximises the lower bound $\mathcal{F}(q(\mathbf{z}, \tilde{\boldsymbol{\theta}}), \boldsymbol{\theta})$ by alternating between E-step and M-step:

- E-step: $q(\mathbf{z}, \tilde{\boldsymbol{\theta}}^{(k)}) = p(\mathbf{z}|\mathbf{x}, \boldsymbol{\theta}^{(k)})$ (might be intractable, where decrease of $\mathcal{L}(\boldsymbol{\theta})$ is not guaranteed)
- M-step: $\boldsymbol{\theta}^{(k+1)} = \arg \max_{\boldsymbol{\theta}} \mathcal{F}(q(\mathbf{z}, \tilde{\boldsymbol{\theta}}^{(k)}), \boldsymbol{\theta})$

In the form of the auxiliary function

- E-step: Set the component posterior to $P(c_m|\mathbf{x}_i, \boldsymbol{\theta}^{(k)})$ or the latent variable posterior to $p(\mathbf{z}|\mathbf{x}_i, \boldsymbol{\theta}^{(k)})$ using the current parameters.
- M-step: Maximise auxiliary function for new parameters $\boldsymbol{\theta}^{(k+1)} = \arg \max_{\boldsymbol{\theta}} \mathcal{Q}(\boldsymbol{\theta}^{(k)}, \boldsymbol{\theta})$.

which yields the sequence of inequalities for iteration $k + 1$:

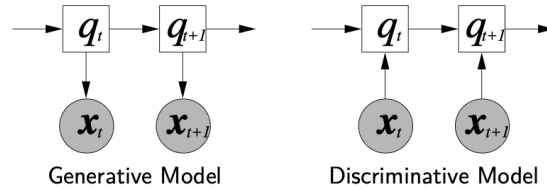
$$\mathcal{L}(\boldsymbol{\theta}^{(k+1)}) \geq \mathcal{F}(q(\mathbf{z}, \tilde{\boldsymbol{\theta}}^{(k+1)}), \boldsymbol{\theta}^{(k+1)}) \geq \mathcal{F}(q(\mathbf{z}, \tilde{\boldsymbol{\theta}}^{(k)}), \boldsymbol{\theta}^{(k)}) = \mathcal{L}(\boldsymbol{\theta}^{(k)})$$

Mean-field approximation : For a distribution $q(\mathbf{z}, \tilde{\boldsymbol{\theta}})$, the mean-field approximation factorises it as $q(\mathbf{z}, \tilde{\boldsymbol{\theta}}) = \prod_i q_i(\mathbf{z}_i, \tilde{\boldsymbol{\theta}}_i)$.

3.2 Sequence Models

Hidden Markov model: a *generative* model that models the joint probability of observations and latent variables. Given a sequence of observations $\mathbf{x}_1, \dots, \mathbf{x}_T$, add discrete latent variables q_t . Assuming conditional independence (Markov property) $P(q_t|q_{t-1}, \dots, q_0) = P(q_t|q_{t-1})$ and $p(\mathbf{x}_t|q_t, \mathbf{x}_{t-1}, \dots, \mathbf{x}_1) = p(\mathbf{x}_t|q_t)$, the likelihood of observations is

$$p(\mathbf{x}_0, \dots, \mathbf{x}_T) = \sum_{\mathbf{q} \in \mathbf{Q}_T} P(q_0) \prod_{t=1}^T P(q_t|q_{t-1}) P(\mathbf{x}_t|q_t)$$



Viterbi approximation: approximate the likelihood of observations by the most likely sequence of latent variables:

$$p(\mathbf{x}_0, \dots, \mathbf{x}_T) \approx \max_{\mathbf{q} \in \mathbf{Q}_T} p(\mathbf{x}_0, \dots, \mathbf{x}_T, \mathbf{q})$$

Viterbi algorithm: a dynamic programming algorithm to find the most likely sequence of latent variables $\hat{\mathbf{q}} = \arg \max_{\mathbf{q} \in \mathbf{Q}_T} p(\mathbf{x}_0, \dots, \mathbf{x}_T, \mathbf{q})$.

Forward-backward algorithm: Define forward probability term $\alpha_j(t) = \log p(\mathbf{x}_1, \dots, \mathbf{x}_t, q_t = s_j)$ and backward probability term $\beta_j(t) = \log p(\mathbf{x}_{t+1}, \dots, \mathbf{x}_T | q_t = s_j)$, and the posterior probability of latent variable q_t is

$$P(q_t = s_j | \mathbf{x}_1, \dots, \mathbf{x}_T) = \frac{\exp(\alpha_j(t) + \beta_j(t))}{\sum_{k=1}^M \exp(\alpha_k(t) + \beta_k(t))}$$

- **Initialisation:** (LZERO = $\log(0)$)
 $\phi_1(0) = 0.0, \quad \phi_j(0) = \text{LZERO}, 1 < j < N, \quad \phi_1(t) = \text{LZERO}, 1 \leq t \leq T$
- **Recursion:**
for $t = 1, \dots, T$
for $j = 2, \dots, N - 1$

$$\phi_j(t) = \max_{1 \leq k < N} \{ \phi_k(t-1) + \log(a_{kj}) \} + \log(b_j(\mathbf{x}_t))$$
- **Termination:**

$$\log(p(\mathbf{x}_1, \dots, \mathbf{x}_T, \hat{\mathbf{q}})) = \max_{1 \leq k < N} \{ \phi_k(T) + \log(a_{kN}) \}$$

Maximum entropy Markov model: a *discriminative* model that directly models the posterior probability of latent variables given observations:

$$P(q_0, \dots, q_T | \mathbf{x}_1, \dots, \mathbf{x}_T) = \frac{1}{Z} \exp \left(\sum_{i=1}^D \lambda_i f_i(q_0, \dots, q_T, \mathbf{x}_1, \dots, \mathbf{x}_T) \right)$$

Linear chain conditional random field (CRF): a maximum entropy Markov model that assume conditional independences similar to HMM, where the posterior probability is

$$P(q_0, \dots, q_T | \mathbf{x}_1, \dots, \mathbf{x}_T) = \frac{1}{Z} \exp \left(\sum_{t=1}^T \left(\sum_{i=1}^{D_t} \lambda_i^t f_i(q_{t-1}, q_t) + \sum_{i=1}^{D_a} \lambda_i^a f_i(q_t, \mathbf{x}_t) \right) \right)$$

where D_t is the number of transition style features and D_a is the number of observation (acoustic) style features. Extending conditional independences to the previous state gives $P(q_0, \dots, q_T | \mathbf{x}_1, \dots, \mathbf{x}_T) = \frac{1}{Z} \exp \left(\sum_{t=1}^T \sum_{i=1}^D \lambda_i f_i(q_t, q_{t-1}, \mathbf{x}_t) \right)$.

General sequence CRF: incorporate cliques between variables, where the posterior probability is

$$p(q_0, \dots, q_T | \mathbf{x}_1, \dots, \mathbf{x}_T) = \frac{1}{Z} \exp \left(\sum_{t=1}^T \sum_{c \in C} \lambda_c^\top \mathbf{f}(q_{c_t}, \mathbf{x}_1, \dots, \mathbf{x}_T, t) \right)$$

To train a CRF, maximise the posterior of $q_{1:T}$ given $\mathbf{x}_{1:T}$.

4 Deep Learning

4.1 Neural Networks

Universal approximation theorem: with enough hidden units, a shallow neural network can describe any continuous function on a compact subset of $\mathbb{R}^D$ to arbitrary precision.

Activation functions: add non-linearity to the network, which is necessary for the network to learn complex functions. If the activation function is linear, the network can be reduced to a single layer (no backpropagation needed).

Hyperparameters of a neural network: number of layers = depth of network, number of hidden units per layer = width of network. A deep network is preferable to a single

wide layer. The total number of parameters should be matched to the quantity of training data. Too many parameters can lead to overfitting, while too few parameters can lead to underfitting (however may improve generalisation).

4.2 Training Neural Networks

Dimensionality of derivatives: Any function $f(x) \rightarrow y$ where $x \in \mathbb{R}^{M \times N \times P}, y \in \mathbb{R}^{R \times S \times T}$ has a Jacobian $f'(x) \rightarrow J$ where $J \in \mathbb{R}^{R \times S \times T \times M \times N \times P}$. The large intermediates Jacobians can be avoided by reverse derivative using Vector-Jacobian Products.

Vector-Jacobian Product: Defined as

$$\vec{f}(x; v) = v \cdot f'(x)$$

Basic rule: every line $p, q = f(x, y)$ in the forward pass becomes $x', y' = \vec{f}(x, y; p', q')$.

The multiply-and-add function $\mu(A, x, b) = Ax + b$ have VJP $\vec{\mu}(A, x, b; v) = (vx^\top, A^\top v, v)$.

Stochastic gradient descent: use a mini-batch of data to compute the gradient and update parameters to introduce noise to escape local minima and saddle points:

$$\phi_{t+1} \leftarrow \phi_t - \alpha \sum_{i \in \mathcal{B}_t} \frac{\partial l_i[\phi_t]}{\partial \phi_t}$$

The mini-batches are sampled from the dataset without replacement, and one pass through the data is an *epoch*.

Momentum: weighted sum of current gradient and previous gradient

$$\begin{aligned} m_{t+1} &\leftarrow \beta m_t + (1 - \beta) \sum_{i \in \mathcal{B}_t} \frac{\partial l_i[\phi_t]}{\partial \phi_t} \\ \phi_{t+1} &\leftarrow \phi_t - \alpha m_{t+1} \end{aligned}$$

Adaptive moment estimation (Adam): measure mean and pointwise squared gradient with momentum

$$\begin{aligned} m_{t+1} &\leftarrow \beta m_t + (1 - \beta) \frac{\partial L[\phi_t]}{\partial \phi_t} \\ v_{t+1} &\leftarrow \gamma v_t + (1 - \gamma) \left(\frac{\partial L[\phi_t]}{\partial \phi_t} \right)^2 \\ \tilde{m}_{t+1} &\leftarrow \frac{m_{t+1}}{1 - \beta^{t+1}} \\ \tilde{v}_{t+1} &\leftarrow \frac{v_{t+1}}{1 - \gamma^{t+1}} \\ \phi_{t+1} &\leftarrow \phi_t - \alpha \frac{\tilde{m}_{t+1}}{\sqrt{\tilde{v}_{t+1}} + \epsilon} \end{aligned}$$

He initialisation: The variance of $y = \phi(Ax)$, where ϕ is the ReLU and $A \in \mathbb{R}^{M \times N}$ is roughly

$$\sigma_y^2 \leq \frac{N}{2} \sigma_A^2 \sigma_x^2$$

So initialise A with variance $\sigma_A^2 = \frac{2}{N}$ to prevent vanishing or exploding gradients.

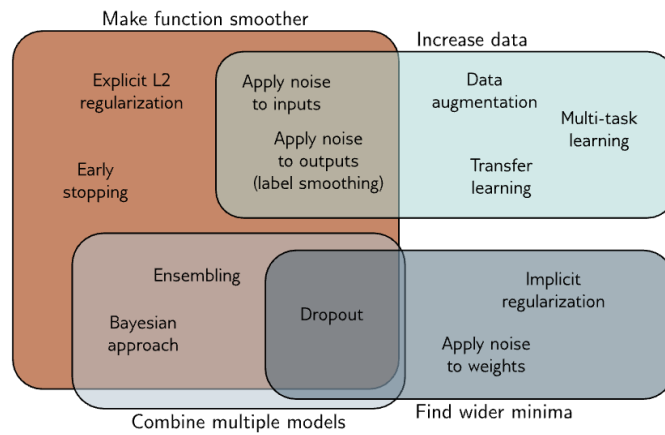
(The output variances should always be matched to the input variances.)

Newton's method: update parameters by the Hessian matrix H and the gradient g :

$$\theta_{t+1} \leftarrow \theta_t - \frac{1}{2} H^{-1} g$$

This only works when H is positive definite. Damped Newton's method adds a damping term λ : $\theta_{t+1} \leftarrow \theta_t - \frac{1}{2} (H + \lambda I)^{-1} g$ to make $H + \lambda I$ positive definite.

Regularisation: methods to reduce the generalisation gap due to overfitting to training data or model being unconstrained in areas with no training examples.



4.3 Architectures

Layer norm: normalise a vector $z \in \mathbb{R}^D$ as

$$N(z)_i = \frac{z_i - \text{mean}(z)}{\text{std}(z) + \epsilon}$$

where $\text{mean} = \frac{1}{D} \sum_{i=1}^D z_i$ and $\text{std} = \sqrt{\frac{1}{D} \sum_{i=1}^D (z_i - \text{mean}(z))^2}$.

Batch norm: normalise a batch of vectors $z \in \mathbb{R}^{B \times D}$ as

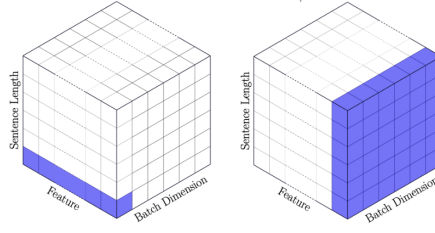
$$N(z)_{i,j} = \frac{z_{i,j} - \text{mean}(z)_j}{\text{std}(z)_j + \epsilon}$$

where $\text{mean}(z)_j = \frac{1}{B} \sum_{i=1}^B z_{i,j}$ and $\text{std}(z)_j = \sqrt{\frac{1}{B} \sum_{i=1}^B (z_{i,j} - \text{mean}(z)_j)^2}$.

Translation invariance: a function F is invariant if for all transformations ϕ , $F(X * \phi) = F(X)$.

General assumptions on transformations: 1) there is always an inverse ϕ^{-1} ; 2) a composition of two transformations can be represented as a single transformation $\phi_1 * \phi_2$.

Convolution kernel: a kernel applied across all positions to weigh neighbouring inputs for each output – translation invariance, with parameters:



- *Stride*: shift by k positions for each output. Larger stride decreases size of output relative to input.
- *Kernel size*: the number of inputs for each output. Larger kernel size combines information from a larger area.
- *Dilated convolution*: intersperse kernel values with zeros. Larger dilation combines information from a larger area with fewer parameters.
- *Channel*: apply several convolution kernels to the same input to produce several outputs (feature maps). Larger number of channels allows the model to learn more features.

Downsampling: introduces an information bottleneck. Methods include: strided convolution, max pooling, mean pooling.

Upsampling: expand feature maps to a larger size. Methods include: duplicate, max upsampling, bilinear interpolation.

Residual connection: add the input to the output of a layer to avoid vanishing or exploding gradients and smooths the loss landscape.

Lottery ticket hypothesis: a small subnetwork within a larger network can be trained in isolation to achieve comparable performance to the original network.

5 Deep Learning for Sequences

5.1 Sequence I/O Pair Modelling

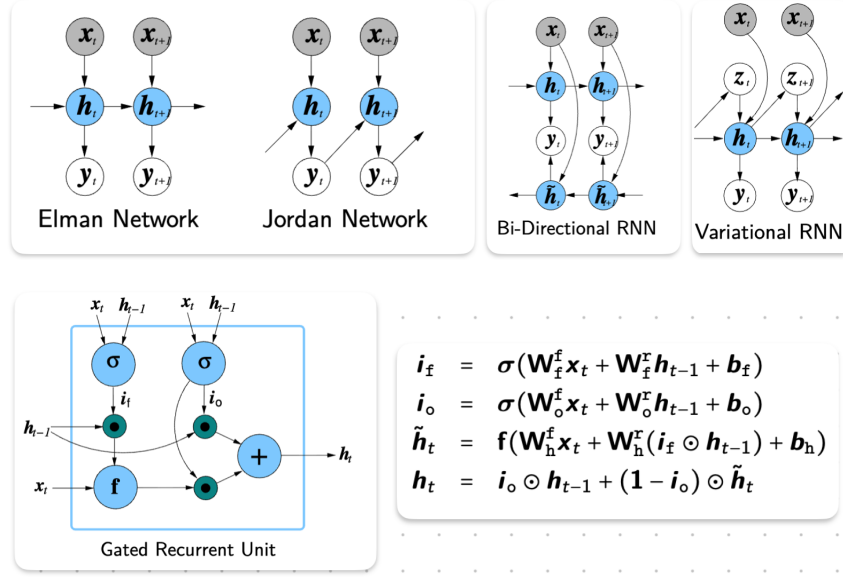
Recurrent neural network (RNN): network with a history vector h_t used to encode causal memory

$$\begin{aligned}
 h_t &= f^h(h_{t-1}, x_t) \text{ (Elman network)} \\
 \text{or } h_t &= f^h(y_{t-1}, x_t) \text{ (Jordan network)} \\
 y(x_{1:t}) &= f^y(h_t)
 \end{aligned}$$

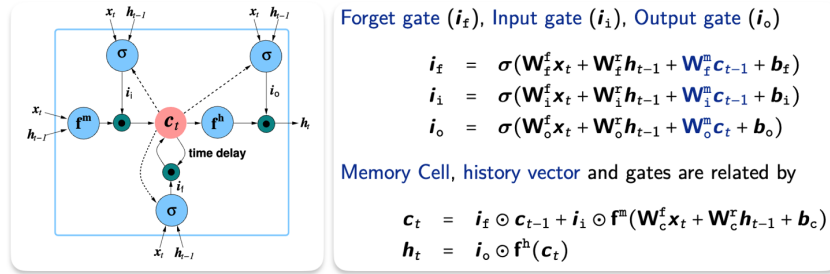
Variational RNN introduce a latent variable sequence $z_{1:T}$

$$p(y_t | x_{1:t}) \approx \int p(y_t | \underbrace{h_{t-1}, z_t, x_t}_{\text{determines } h_t}) p(z_t | h_{t-1}) dz_t$$

Gated recurrent unit (GRU): using forget gate i_f and output gate i_o to control the contribution of last history h_{t-1} and current input x_t to the current history vector h_t .



Long short-term memory (LSTM): using forget gate i_f , input gate i_i and output gate i_o to update memory cell c_t ('long-term memory') and history vector h_t ('short-term memory'). The memory cell weight matrices W_f^m, W_i^m, W_o^m are diagonal such that individual cell elements can be analysed explicitly.



Highway connections: gate input and output of a layer (rather than direct addition in residual connection).

5.2 Sequence to Target

Averaging: compute the average of states $h_1, \dots, h_T$, where $h_t = \mathcal{F}(x_t)$.

RNN encoding: use the last state h_T , where $h_t = \mathcal{F}(h_{t-1}, x_t)$.

Attention mechanism: compute a weighted average of states $h_1, \dots, h_T$ to produce the target c given a query q :

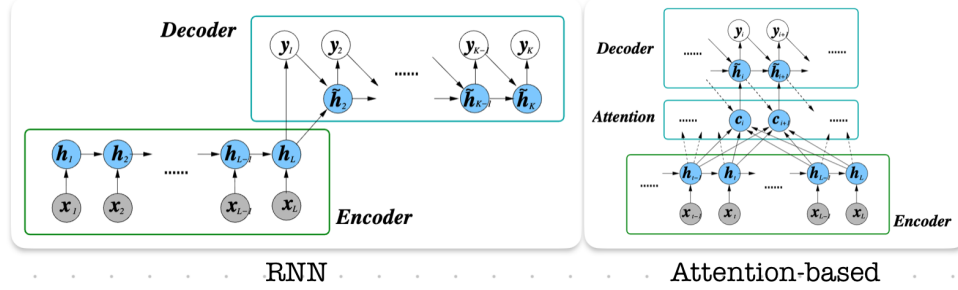
$$c = \sum_{t=1}^T \alpha_t h_t, \text{ where } \alpha_t = \frac{\exp(\mathcal{F}(h_t, \tilde{h}))}{\sum_{i=1}^T \exp(\mathcal{F}(h_i, \tilde{h}))}, \tilde{h} = \mathcal{F}_q(q)$$

Standard forms of attention:

- *Dot-product attention:* $\mathcal{F}(h_t, \tilde{h}) = h_t^\top W_{xq} \tilde{h}$
- *Additive attention:* $\mathcal{F}(h_t, \tilde{h}) = w^\top \tanh(W_x h_t + W_q \tilde{h})$

5.3 Sequence to Sequence

Encoder-decoder architecture: use an encoder to encode the input sequence into a fixed-length vector(s). and use a decoder to decode the vector into an output sequence.



$$p(\mathbf{y}_{1:K}|\mathbf{x}_{1:L}) = \prod_{i=1}^K p(y_i|\mathbf{y}_{1:i-1}, \mathbf{x}_{1:L}) \approx \prod_{i=1}^K p(y_i|y_{i-1}, \tilde{\mathbf{h}}_{i-1}, \mathbf{c}_i) \approx \prod_{i=1}^K p(y_i|\tilde{\mathbf{h}}_i)$$

For RNN, $\mathbf{c}_i = \mathbf{h}_L$.

For attention-based model, $\mathbf{c}_i = \sum_{\tau=1}^L \alpha_{i\tau} \mathbf{h}_\tau$, where $\alpha_{i\tau} = \frac{\exp(\mathcal{F}(\mathbf{h}_\tau, \tilde{\mathbf{h}}_{i-1}))}{\sum_{j=1}^L \exp(\mathcal{F}(\mathbf{h}_j, \tilde{\mathbf{h}}_{i-1}))}$. The previous translation history vector $\tilde{\mathbf{h}}_{i-1}$ is used as a query, where $\tilde{\mathbf{h}}_i = \mathcal{F}_q(y_{i-1}, \tilde{\mathbf{h}}_{i-1}, \mathbf{c}_i)$ encodes the translation history up-to $i-1$ and generates the output y_i .

Alternative architecture: Use the current translation history vector $\tilde{\mathbf{h}}_i = \mathcal{F}_q(y_{i-1}, \tilde{\mathbf{h}}_{i-1})$ and the context $\mathbf{c}_i$. Use $\hat{\mathbf{h}}_i = \begin{bmatrix} \mathbf{c}_i \\ \tilde{\mathbf{h}}_i \end{bmatrix}$ to generate the output y_i (Tripos 2023).

Word embeddings: represent words as vectors

$$\underbrace{w_i}_{\text{word}} \rightarrow \underbrace{t_i}_{\text{one-hot encoding}} \rightarrow \underbrace{Wt_i}_{\text{embedding}} = \mathbf{x}_i$$

Transformer architecture:

- *Positional encoding:* enable the model to consider the order of sequence elements during attention computation.
- *Scaled dot-product multi-head attention:* multiple attention heads allow the model to attend to different parts of the input sequence simultaneously to capture different types of relationships and provide richer and more versatile representations of the sequence elements.

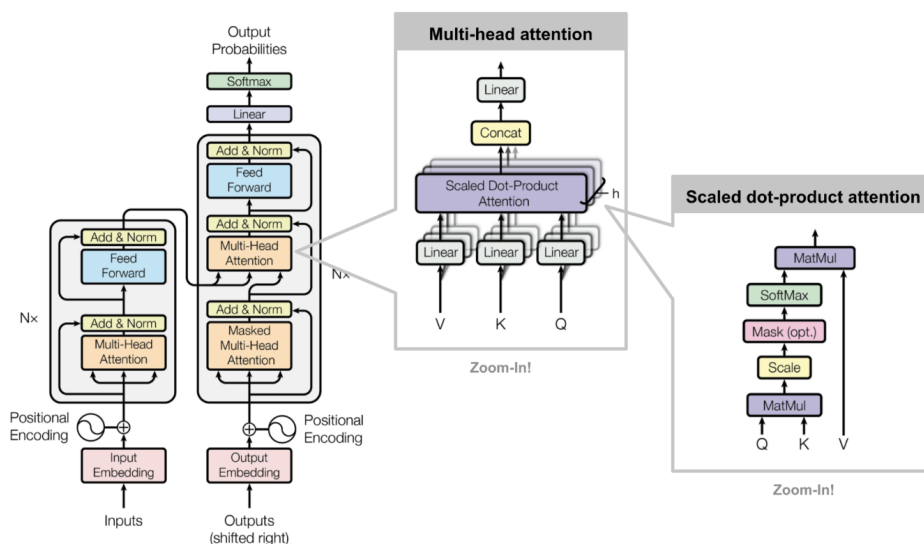
$$\alpha_{\tau i}^{(j)} = \frac{1}{Z} \exp \left(\frac{\overbrace{(W_q^{(j)} \mathbf{x}_\tau)^\top (W_k^{(j)} \mathbf{x}_i)}^{\text{query key}}}{\sqrt{d_k^{(j)}}} \right)$$

$$\mathbf{c}_i^{(j)} = \sum_{\tau=1}^L \alpha_{\tau i}^{(j)} \overbrace{W_v^{(j)} \mathbf{x}_\tau}^{\text{value}}$$

$$\mathbf{c} = [\mathbf{c}_i^{(1)\top}, \dots, \mathbf{c}_i^{(H)\top}]^\top$$

Three types of attentions are used:

- *Multi-head self-attention*: use the query, key and value from the same sequence to compute the relationship between all words in the sequence in parallel, helps the model to focus on relevant parts of the sequence.
- *Masked multi-head attention*: mask the attention weights $\alpha_{\tau i}^{(j)}$ for $\tau > i$ (by setting them to $-\infty$ before applying softmax) to prevent the model from attending to future inputs.
- *Multi-head cross-attention*: use values and key from the encoder and query from the (masked) decoder.
- *Feedforward network*: apply to the output of multi-head attention.
- *Add and norm*: add the input to the output of each layer (= residual connection) and apply layer norm.
- *Output layer*: use a linear projection to map the output of the decoder to the vocabulary size, and apply softmax to get an posterior distribution of next word.



The computation cost is $O(L^2)$ where L is the sequence length.

RNNs vs Transformers:

- **Parallelism**: Transformers process all elements of the sequence simultaneously, while RNNs process them sequentially. Hence transformers are faster for long sequences by using GPU acceleration.
- **Sequence length**: Transformers can handle long-range dependencies efficiently due to self-attention, while RNNs struggle with vanishing gradients.
- **Scalability**: Transformers scale better with hardware acceleration, while RNNs are limited by their sequential nature. However, the computation cost is $O(L^2)$ for transformers and $O(L)$ for RNNs.
- **Biases**: Transformers allow all elements of the sequence to interact, while RNNs have a bias towards the end of the sequence.

Greedy decoding: At each step, choose the most likely output $\hat{y}_i = \arg \max_{y_i} p(y_i | \hat{\mathbf{y}}_{1:i-1}, \mathbf{x}_{1:L})$. Repeat until the end of sentence token `eos` is generated.

Teacher-forcing: Use the true output history $\mathbf{y}_{1:j-1}$ as input to the decoder at step j during training. The training criterion is the *negative cross-entropy*:

$$\mathcal{L} = \sum_{i=1}^N \sum_{j=1}^J \sum_{k=1}^V \delta(w_k, y_{ij}) \log P(w_k | \mathbf{x}_{1:L}, \mathbf{y}_{i,1:j-1}, \boldsymbol{\theta})$$

Advantage: The updates can be done in parallel.

Limitation: At inference time, there is a mismatch between the training history and the generated history, which can lead to error accumulation.

6 Ensemble Methods

Binary ensemble classifier: select the class with the most votes from a set of binary classifiers (majority voting). The probability of error is

$$P(\text{error}) = \sum_{i=\lceil \frac{M}{2} \rceil}^M \binom{M}{i} p^i (1-p)^{M-i}$$

Bayesian ensemble:

$$\hat{\omega} = \arg \max_{\omega} \left\{ \sum_{\mathcal{M}} \int P(\omega | \mathbf{x}^*, \mathcal{M}, \boldsymbol{\theta}) p(\mathcal{M}, \boldsymbol{\theta} | \mathcal{D}) d\boldsymbol{\theta} \right\}$$

where $\mathcal{M}$ is a model topology and $\boldsymbol{\theta}$ is the parameters of the model. The Monte-Carlo approximation is

$$\hat{\omega} = \arg \max_{\omega} \left\{ \frac{1}{M} \sum_{m=1}^M P(\omega | \mathbf{x}^*, \boldsymbol{\theta}^{(m)}) \right\}, \text{ where } \boldsymbol{\theta}^{(m)} \sim p(\mathcal{M}, \boldsymbol{\theta} | \mathcal{D})$$

Monte-Carlo dropout: generate ensemble by randomly de-activating nodes during training.

Random network initialisation: given a topology $\mathcal{M}$, initialise multiple network parameters as $\tilde{\boldsymbol{\theta}}^{(m)} \sim p(\boldsymbol{\theta} | \mathcal{M})$. Train each network using the same data.

Posterior combination: a simple way to combine the predictions of multiple models is to take the average of their posterior probabilities:

$$P(\omega | \mathbf{x}^*) = \frac{1}{M} \sum_{m=1}^M P(\omega | \mathbf{x}^*, \boldsymbol{\theta}^{(m)})$$

This assumes that each of the models is approximately a draw from the parameter distribution.

Teacher-student training: train a student model to mimic the predictions of a teacher model (rather than the true labels) by minimising the cross-entropy loss

$$\mathcal{F}_{ts} = - \sum_{i=1}^N \sum_{\omega} P(\omega | \mathbf{x}_i, \boldsymbol{\theta}_t) \log P(\omega | \mathbf{x}_i, \boldsymbol{\theta}_s)$$

The single teacher model can be replaced by an ensemble of models.

Mixture of experts: use a gating function $P(e_m | \mathbf{x}^*, \boldsymbol{\theta})$ to combine the predictions of multiple models

$$P(\omega | \mathbf{x}^*) = \sum_{m=1}^M P(e_m | \mathbf{x}^*, \boldsymbol{\theta}) P(\omega | \mathbf{x}^*, \boldsymbol{\theta}, e_m)$$

where each expert focus on different aspects of the data.

Switch transformer: use a router (equivalent to the gating function) to select between different feedforward networks.

7 Support Vector Machines

Maximum margin classifier:

- When the data is linearly separable, a maximum margin classifier (aka *hard margin support vector machine*) finds the hyperplane that maximises the margin – the distance to the closest point in each class.
- The data points on the margin are called *support vectors* – they determine the position and orientation of the hyperplane.
- By maximising the margin, SVM favours better generalization to unseen data.
- The magnitude of the margin is

$$\frac{w^\top (x_+ - x_-)}{2\|w\|} = \frac{1}{\|w\|}$$

where x_+ and x_- are the support vectors of the positive and negative class respectively.

Optimisation of maximum margin classifier: Minimise $\frac{1}{2}\|w\|^2$ subject to $t_n(w^\top x_n + b) \geq 1$ for all n . The Lagrangian is

$$\mathcal{L}(w, b, \mathbf{a}) = \frac{1}{2}\|w\|^2 - \sum_{n=1}^N a_n(t_n(w^\top x_n + b) - 1)$$

Full KKT conditions:

1. Primal feasibility: $t_n(w^\top x_n + b) - 1 \geq 0$
2. Dual feasibility: $a_n \geq 0$
3. Complementary slackness: $a_n(t_n(w^\top x_n + b) - 1) = 0$
4. Stationarity: $w = \sum_{n=1}^N a_n t_n x_n, \sum_{n=1}^N a_n t_n = 0$ (from differentiating w.r.t. w and b)

Data points with $a_n > 0$ are the support vectors, which determine the predictions:

$$y(x) = \sum_{n \in \mathcal{S}} a_n t_n x_n^\top x + b$$

where $\mathcal{S}$ is the set of indexes of support vectors. The bias can be obtained by the constraint $t_n(w^\top x_n + b) = 1$

$$b = \frac{1}{|\mathcal{S}|} \sum_{n \in \mathcal{S}} \left(t_n - \sum_{m \in \mathcal{S}} a_m t_m x_m^\top x_n \right)$$

Optimisation with soft margin: Soft margin allows some data points to be misclassified by introducing slack variables $\xi_1, \dots, \xi_N \geq 0$ in the constraints $t_n(w^\top x_n + b) \geq 1 - \xi_n$ for all n .

Minimise $\frac{1}{2} \|w\|^2 + C \sum_{n=1}^N \xi_n$ subject to $t_n(w^\top x_n + b) \geq 1 - \xi_n$ and $\xi_n \geq 0$ for all n . The Lagrangian is

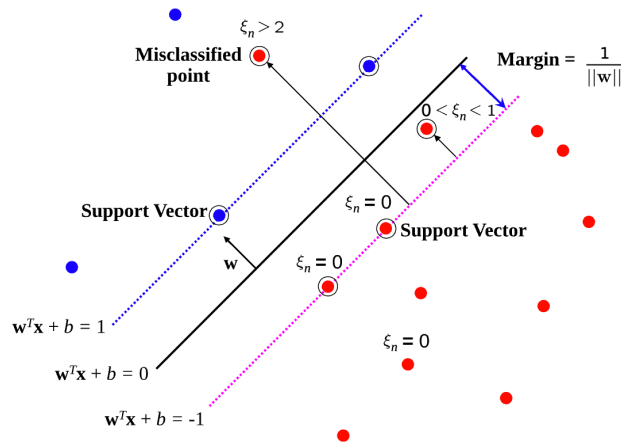
$$L(w, b, \mathbf{a}, \boldsymbol{\mu}) = \frac{1}{2} \|w\|^2 + C \sum_{n=1}^N \xi_n - \sum_{n=1}^N a_n (t_n(w^\top x_n + b) - 1 + \xi_n) - \sum_{n=1}^N \mu_n \xi_n$$

with Lagrange multipliers $a_n = C - \mu_n \geq 0$ and $\mu_n \geq 0$.

The regularisation parameter $C > 0$ controls the trade-off between margin size and misclassification rate. Small C allows more misclassification and larger margin (useful when there are many outliers) and vice versa.

Full KKT conditions:

1. Primal feasibility: $t_n(w^\top x_n + b) \geq 1 - \xi_n$ and $\xi_n \geq 0$
2. Dual feasibility: $a_n \geq 0$ and $\mu_n \geq 0$
3. Complementary slackness: $a_n(t_n(w^\top x_n + b) - 1 + \xi_n) = 0$ and $\mu_n \xi_n = 0$
4. Stationarity: $w = \sum_{n=1}^N a_n t_n x_n$, $\sum_{n=1}^N a_n t_n = 0$ (from differentiating w.r.t. w and b), and $a_n = C - \mu_n$ (from differentiating w.r.t. ξ_n)



Non-linear max-margin classifier: replace feature vectors $\mathbf{x}_n$ with a non-linear mapping $\phi(\mathbf{x}_n) = [\phi_1(\mathbf{x}_n), \dots, \phi_M(\mathbf{x}_n)]^\top$. The *Gram matrix* consists of inner products of feature vectors, where $K_{nm} = \phi(\mathbf{x}_n)^\top \phi(\mathbf{x}_m)$.

Kernel function: $k(\mathbf{x}_n, \mathbf{x}_m) = \phi(\mathbf{x}_n)^\top \phi(\mathbf{x}_m)$ can be used to compute the dot products without explicitly computing the feature vectors.

Advantages of kernel:

- Avoiding explicitly computing the feature vectors allows very high-dimensional feature spaces.
- Can be used to obtain non-linear version of linear machine learning algorithms that can be written in terms of dot products of data points.

Mercer's condition: $k(\cdot, \cdot)$ is a valid kernel function if

1. k is symmetric
2. the Gram matrix K is positive semi-definite for any set of input vectors.

Kesler's construct: add constraints to enforce output of correct classifier than the others

$$w_{t_n}^\top \phi(\mathbf{x}_n) + b_{t_n} - (w_j^\top \phi(\mathbf{x}_n) + b_j) > 0, \text{ for all } j \neq t_n$$

Merge all the weights into a single vector $\bar{w}^\top = [w_1^\top, b_1, \dots, w_K^\top, b_K]$. The constraints can be written in the form of a binary classifier (... denotes zeros)

$$\bar{w}^\top \bar{\phi}(\mathbf{x}_n)^{(t_n j)} > 0, \text{ where } \bar{\phi}(\mathbf{x}_n)^{(t_n j)\top} = [\dots, \underbrace{\phi(\mathbf{x}_n)^\top}_{\text{element } t_n}, 1, \dots, \underbrace{-\phi(\mathbf{x}_n)^\top}_{\text{element } j}, -1, \dots]$$

The dimensionality of feature space is Kd and the number of constraints (effective number of data points) is $(K - 1)N$.

Perceptron algorithm:

1. Initialise $w = 0$ and $b = 0$.
2. For each data point, if it is misclassified, update the weights and bias

$$w \leftarrow w + t_n \mathbf{x}_n \text{ and } b \leftarrow b + t_n.$$

3. Repeat until all data points are correctly classified.

Classification is based on $y(\mathbf{x}^*) = w^\top \mathbf{x}^* + b = \sum_{n=1}^N \alpha_n t_n \mathbf{x}_n^\top \mathbf{x}^* + b$, where α_n is the number of times the data point n is misclassified.

The kernelised version is:

1. Initialise $\alpha_n = 0$ for all n and $b = 0$.
2. For each data point, if it is misclassified, update the weights and bias:

$$\alpha_n \leftarrow \alpha_n + 1 \text{ and } b \leftarrow b + t_n.$$

3. Repeat until all data points are correctly classified.

Classification is based on $y(\mathbf{x}^*) = \sum_{n=1}^N \alpha_n t_n k(\mathbf{x}_n, \mathbf{x}^*) + b$. α_n represent the number of times the data point n is misclassified.

8 Kernels for Structured Data

String kernel: Given a list of substrings $s_1, \dots, s_m$, the feature $\phi_s(x)$ indicates the occurrence of s in x . The kernel between two strings x and x' is defined as

$$k(x, x') = \sum_s \phi_s(x) \phi_s(x')$$

Examples:

- *Gap-weighted kernel:* $\phi_s(x) = \lambda^n$ where $0 < \lambda < 1$.
- *k-spectrum kernel:* $\phi_s(x)$ is the number of occurrences of length- k sequence s in x . When $k = 1$, it becomes the bag-of-words kernel.

Random-walk graph kernel: A k -length walk in a graph is defined as a sequence of k edges. The number of k -length walks between nodes i and j is given by $[A^k]_{ij}$, where A is the adjacency matrix of the graph.

The random-walk kernel $k(G, G')$ counts the number of common walks in two graphs G and G' . Defined as $k(G, G') = [I - \lambda A \otimes A']^{-1}$ where A and A' are the adjacency matrices of G and G' respectively, $\otimes$ is the Kronecker product, and λ is a decay factor.

Weisfeiler-Lehman graph kernel: Repeat for multiple iterations, for each graph in the dataset: For each node in a graph, create a set with labels of adjacent vertices, sort the label set and add vertex label as a prefix, then compress this label sequence into a unique value. The kernel is the bag-of-words kernel on both original nodes and compressed nodes.

Fisher kernel: Train a generative model $p(x|\theta)$ and define the feature vector as the score function at maximum likelihood estimate $\phi(x_n) = \nabla_{\theta} \log p(x_n|\theta)|_{\theta_{\text{MLE}}}$.